Control de acceso por edad con punteros y desreferenciación en C++

Alexis Maier

E.E.S.T. N.º 1 «Eduardo Ader», Vicente López

Laboratorio de Programación (LPR) — 5.º año 3.ª división, grupos A-B

Profesor: Mansilla Muñoz York Elías

21 de octubre de 2026

v1.0.0

Control de acceso por edad con punteros y desreferenciación en C++

Introducción

La memoria RAM de una computadora se organiza como casilleros numerados, cada uno con una dirección física única que se expresa en formato hexadecimal. Un puntero es una variable especial que almacena esa dirección; desreferenciarlo con el operador asterisco permite leer o modificar el valor del casillero al que apunta.

En esta actividad se analizó y ejecutó un sistema de control de acceso por edad que captura el dato directamente en la memoria de la variable original a través de un puntero. El informe explica el funcionamiento del código, muestra la evidencia de ejecución con direcciones hexadecimales reales y responde el cuestionario técnico.

Desarrollo

Análisis del código

La declaración `int* p = &edad;` usa el operador de dirección `&` para obtener la ubicación de `edad` y la guarda en el puntero `p`. La instrucción `cin >> *p;` guarda el número ingresado en el casillero de `edad`. La condición `*p >= 18` evalúa el valor almacenado: si es verdadera se imprime ACCESO APROBADO y, si no, ACCESO RECHAZADO. En ambos casos se muestra `*p` (el valor) y `p` (la dirección hexadecimal).

Compilación y ejecución

Se compiló con `g++ src/main.cpp -o src/cadenaspunteros.exe` y se probó con 25 años (acceso aprobado) y 15 años (acceso rechazado). La dirección es siempre la misma dentro de una ejecución, porque `edad` ocupa un único casillero.

```
PROBLEMS   OUTPUT   DEBUG CONSOLE          TERMINAL — cadenaspunteros.exe

PS C:\Users\Alexis\Desktop\Actividades-2.0> g++ src/main.cpp -o src/cadenaspunteros.exe
PS C:\Users\Alexis\Desktop\Actividades-2.0> .\src\cadenaspunteros.exe
====================================================
 CONTROL DE ACCESO RAM - ESTUDIANTE: Alexis Maier
====================================================
=> Ingrese su edad: 25

--- ANALIZANDO ACCESO SEGURO EN MEMORIA ---
[ACCESO APROBADO] El usuario es mayor de edad.
Edad registrada: 25 anos.
Direccion fisica en RAM Hexadecimal: 0xb450fff804
====================================================
PS C:\Users\Alexis\Desktop\Actividades-2.0> .\src\cadenaspunteros.exe
====================================================
 CONTROL DE ACCESO RAM - ESTUDIANTE: Alexis Maier
====================================================
=> Ingrese su edad: 15

--- ANALIZANDO ACCESO SEGURO EN MEMORIA ---
[ACCESO RECHAZADO] Menor de edad.
Edad registrada: 15 anos.
Direccion fisica en RAM Hexadecimal: 0xb54f5ffbf4
====================================================
PS C:\Users\Alexis\Desktop\Actividades-2.0> _
```

Figura 1. Ejecución del programa mostrando las direcciones hexadecimales.

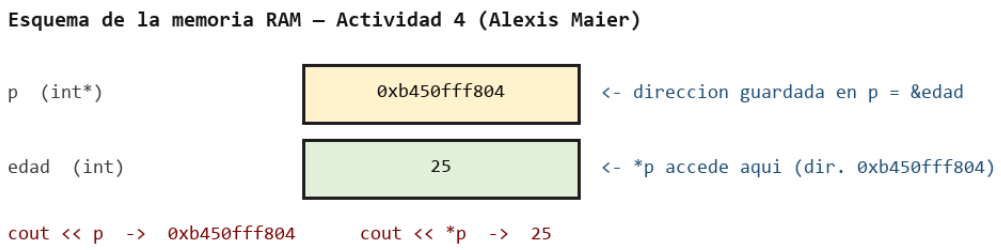


Figura 2. Esquema de la RAM: el puntero p guarda la dirección de la variable edad.

Cuestionario técnico de control

Pregunta 1: ¿Qué valor imprime cout << p; comparada con cout << *p;?

`cout << p;` imprime la dirección de memoria que guarda el puntero, en hexadecimal (en la ejecución documentada, `0xb450fff804`). `cout << *p;` desreferencia el puntero e imprime el valor contenido en esa dirección, es decir, la edad ingresada (por ejemplo 25). El operador `&` obtiene la ubicación y el `*` accede al contenido.

Pregunta 2: ¿Por qué es obligatorio desreferenciar el puntero (*p) al usar cin para modificar el valor de la variable original?

Porque `p` contiene una dirección y `*p` es la variable a la que apunta. Si se escribe `cin >> p;` se intenta leer un valor para la propia dirección, no para `edad`, y el compilador lo rechaza. Se comprobó compilando esa línea: `err_cin.cpp:6:8: error: no match for 'operator>>' (operand types are 'std::istream' {aka 'std::basic_istream<char>'} and 'int*')`. Con `cin >> *p;` el dato se guarda directamente en el casillero de `edad`, que es lo que se busca.

Pregunta 3: ¿Por qué el sistema operativo utiliza la notación hexadecimal en lugar de decimal para representar las direcciones de memoria?

Porque el hexadecimal (base 16) es una forma compacta de escribir números binarios: cada dígito hexadecimal equivale exactamente a 4 bits, por lo que un byte son 2 dígitos y una dirección de 64 bits se escribe con 16 dígitos en lugar de 64 ceros y unos. Además, la conversión entre binario y hexadecimal es directa, mientras que con base 10 no coincide con los límites de bits. El prefijo `0x` indica que el número está en base 16.

Conclusión

El uso de punteros permite trabajar directamente con la memoria RAM: `&` entrega la dirección, `*` accede al contenido, y mezclar ambos conceptos produce errores que el compilador detecta. La actividad permitió ver direcciones reales de memoria, entender por qué `cin` necesita `*p` y relacionar el formato hexadecimal con la representación binaria de la computadora.

Referencias

cplusplus.com. (s. f.). *Pointers*. https://cplusplus.com/doc/tutorial/pointers/

Microsoft. (s. f.). *Operadores de dirección e indirección en C++*. https://learn.microsoft.com/es-es/cpp/cpp/

GeeksforGeeks. (s. f.). *Pointers in C++*. https://www.geeksforgeeks.org/cpp-pointers/

Mansilla Muñoz, Y. E. (2026). *Actividad 4: Manipulación de cadenas y punteros en C/C++* [Guía de actividad]. E.E.S.T. N.º 1 «Eduardo Ader».